Simulation Godot du PiCar

Nov 11, 2026 · @Raph Raphael

Lire d'abord

La simulation Godot est un faux PiCar qui roule sur un faux circuit et qui est piloté par le vrai code C++ du robot : le FSM ne sait pas s'il parle à Godot ou à la voiture réelle. Godot envoie 50 fois par seconde ce que les capteurs « voient », le FSM répond avec une puissance moteur et un angle de roues, et Godot déplace la voiture en conséquence.

Ce document explique tout depuis zéro, pour quelqu'un qui n'a jamais ouvert Godot. Il sert aussi de référence : on peut le commenter, le modifier et l'exporter en Word, PDF ou Markdown.

Par où commencer

- Faire tourner la simulation : section « Lancer la simulation ».
- Comprendre comment Godot est construit : « Godot de zéro », puis « Les capteurs ».
- Comprendre comment le robot décide : « Le cerveau FSM », « Le suiveur de ligne pas à pas » et « L'évitement d'obstacle en 12 étapes ».
- Quelque chose ne marche pas : « Dépannage par symptôme ».

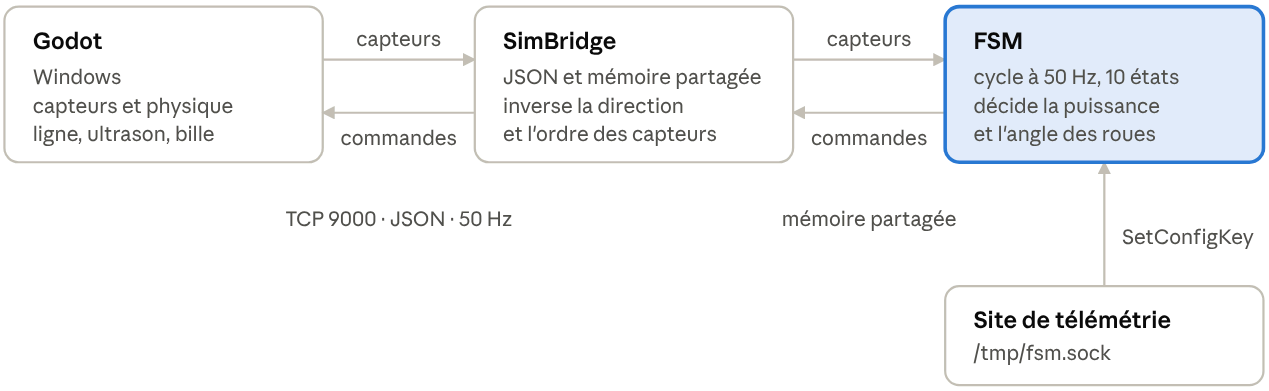
Ce qui est vérifié, et ce qui ne l'est pas

Sujet	État
Évitement du premier obstacle	Réussi 4 fois sur 4 en simulation (droite et gauche, avec et sans mode réaliste)
Évitement du deuxième obstacle	Réussi des deux côtés sans mode réaliste
Arrêt à l'arrivée sans faire tomber la bille	Réussi dans 3 essais sur 3 après correction (2 chutes sur 3 avant)
Arrêt d'urgence	Le robot reste immobile (dérive de 0,000 m) et repart après réarmement
Mode réaliste	Ouvert : le suivi de ligne échoue au premier virage, même sans obstacle
Tour complet	Ouvert : arrêts à tort en cours de parcours depuis la simplification de la machine à états

Sujet	État
Robot réel	Rien n'a été validé : le robot est hors ligne

Vue d'ensemble

Godot ne parle jamais directement au FSM : SimBridge traduit entre les deux



architecture de la simulation · 4 éléments

Godot simule le monde, le FSM décide, et SimBridge traduit entre les deux. Le site de télémétrie modifie les réglages du FSM à chaud par `/tmp/fsm.sock`. Le même FSM tourne sur le vrai robot, où SimBridge est remplacé par les services de moteurs et de capteurs.

Lancer la simulation

Une seule commande lance les services C++ (SimBridge et FSM) dans WSL, puis on ouvre Godot côté Windows et on appuie sur le bouton de démarrage.

1. Dans WSL, depuis le dossier `services/` : `./run_sim.sh`. Le script crée le tube de démarrage `/tmp/peripherals.sock` (un FIFO), puis lance SimBridge et le FSM.
2. Ouvrir le projet `godot/` dans Godot 4.7 et lancer la scène `main.tscn` (touche F5).
3. Godot se connecte à SimBridge sur le port TCP 9000. Dans la fenêtre, les boutons de la scène arment et démarrent le robot.
4. Pour arrêter : Ctrl+C dans WSL, puis fermer Godot.

Option ou touche	Effet
`./run_sim.sh --direction gauche` ou `droite`	Choisit le côté du contournement (variable `AVOID_SIDE`)
Touche `R` dans Godot	Active ou désactive le mode réaliste
Variable `REALISM=1`	Démarre directement en mode réaliste
Variable `REALISM_SEED=<n>`	Fixe le hasard pour rejouer le même parcours
`--optuna --time-scale N --timeout T`	Mode sans fenêtre pour les tests rapides (voir « Tests et résultats »)

Le FSM tourne à la fréquence `HZ` du fichier `fsm.conf`, fixée à 50 Hz. Les gains du suiveur de ligne sont réglés pour cette fréquence : à 100 Hz, l'évitement échouait de façon aléatoire.

Si le robot ne démarre pas, vérifier d'abord que le tube `/tmp/peripherals.sock` existe : `run_sim.sh` refuse de continuer si un fichier ordinaire porte ce nom.

Godot de zéro

Godot organise un monde en arbre de nœuds : chaque objet (sol, voiture, capteur, cube) est un nœud, et un nœud peut porter un script GDScript qui décrit son comportement. Toute la simulation tient dans une seule scène, `main.tscn`.

Nœud	Rôle
`Floor`	Le sol
`CircuitParcours`	Le ruban noir, placé sur la couche de collision 2
`PiCar`	La voiture : script `picar_raycast.gd`, 5 capteurs de ligne, un rayon ultrason
`Bille`	Corps physique (`RigidBody3D`) posé dans une coupelle conique ; script `ball_monitor.gd`
`obstacle`, `obstacle2`	Deux cubes d'environ 15 cm, sur la couche 1, avec collision

Scripts à connaître : `main.gd` (boutons et mode Optuna), `network_client.gd` (lien TCP avec SimBridge), `picar_raycast.gd` (déplacement), `line_follower_raycast.gd` (capteurs de ligne), `ball_monitor.gd` (surveillance de la bille) et `realism.gd` (défauts du mode réaliste).

Couches de collision. Un rayon (`RayCast3D`) ne voit que les objets de ses couches. Le ruban est seul sur la couche 2 : les capteurs de ligne ne regardent que cette couche et ne réagissent donc pas aux cubes. Le rayon ultrason regarde la couche 1 (les obstacles) et ignore le ruban.

Repères. Godot utilise des mètres, l'axe Y vers le haut et le plan du sol en X et Z. L'orientation de la voiture est un angle autour de Y.

Déplacement de la voiture (modèle d'Ackermann). Le robot se comporte comme un vélo : seules les roues avant tournent. Avec un empattement L = 0,14 m, une vitesse v et un angle de roues δ, la voiture tourne à la vitesse angulaire :

```
\omega = \frac{v}{L}\tan\delta
```

Le rayon de virage est R = L / tan δ. Pour δ = 20°, R ≈ 0,38 m ; pour δ = 45°, R = 0,14 m. Le script teste aussi une boîte de collision avant chaque déplacement : la voiture ne traverse pas un cube.

Conversions de commande. La puissance moteur du FSM devient une vitesse par la formule v = (puissance % / 50) × 0,32 m/s : 50 % donne 0,32 m/s. L'angle de roues du FSM (entre −45 et +45) est multiplié par 48°/45 côté Godot.

Les capteurs

Le robot a deux sortes de capteurs : cinq capteurs de ligne sous le pare-chocs avant, et un capteur de distance (ultrason) qui regarde devant. Godot les imite avec des rayons invisibles.

Capteurs de ligne. Un capteur réel est une petite cellule qui mesure la lumière réfléchie : plus la surface est sombre, plus la valeur est élevée. En simulation, chaque capteur lance 15 rayons verticaux répartis sur 19 mm vers le sol. Sa valeur est la fraction de rayons qui touchent le ruban (couche 2) : 0 = tout blanc, 1 = tout noir, 0,4 = environ 6 rayons sur 15.

Le suiveur de ligne comprend où est le ruban en comparant les cinq valeurs. Exemple, avec les poids −2, −1, 0, +1, +2 :

Capteur	1	2	3	4	5
Valeur (noirceur)	0,0	0,4	0,8	0,2	0,0
Poids	−2	−1	0	+1	+2

Barycentre = (0,4×(−1) + 0,8×0 + 0,2×(+1)) / (0,4 + 0,8 + 0,2) = −0,2 / 1,4 ≈ −0,14. Le ruban est donc un peu du côté des poids négatifs ; le FSM corrige en braquant dans ce sens.

Capteur ultrason. Le vrai capteur envoie une onde et mesure le temps d'écho. En simulation, c'est un seul `RayCast3D` de 0,8 m sur la couche 1 : la distance est celle du premier objet touché, convertie en centimètres. Sans obstacle, la mesure est invalide (hors de portée).

Le FSM nettoie la mesure en trois temps : il rejette les valeurs aberrantes, prend la médiane des 5 dernières (qui ignore une valeur isolée fausse), puis lisse avec une moyenne exponentielle de coefficient α = 0,25. Exemple : si la valeur lissée est 50 cm et que la médiane donne 57 cm, la nouvelle valeur est 50 + 0,25 × (57 − 50) = 51,75 cm. Le lissage retarde un peu la détection mais évite les arrêts sur une mesure parasite.

Distance lissée	Réaction du FSM
Moins de 45 cm	Ralentit progressivement (`OBSTACLE_SLOWDOWN_THRESHOLD_CM`)
25 cm ou moins	Seuil de contrôle après l'arrêt : si l'obstacle est encore là, l'évitement démarre
20 cm ou moins	Arrêt (`OBSTACLE_STOP_DIST_CM`, 20 par défaut)

Ce ralentissement avait disparu pendant une refonte : le robot arrivait à pleine vitesse et se coinçait contre le cube. Il est rétabli et couvert par un test unitaire.

Godot ↔ C++

Godot et les services C++ ne partagent pas de mémoire : SimBridge les relie. Godot (Windows) ouvre une connexion TCP sur le port 9000 et échange des lignes de JSON à 50 Hz ; SimBridge traduit ces lignes en messages publiés dans la mémoire partagée (publication/abonnement) que le FSM lit, et fait l'inverse pour les commandes.

Sens	Contenu	Exemple de champs
Godot → SimBridge	État des capteurs	Cinq valeurs de ligne entre 0 et 1, distance ultrason en cm
SimBridge → Godot	Commande du FSM	Puissance moteur en %, angle de roues

Deux conversions sont nécessaires parce que Godot est en miroir de la convention du robot :

- **Direction.** SimBridge inverse l'angle : `-clamp(direction / 45)`, ce qui donne une valeur entre −1 et +1 pour Godot.
- **Ordre des capteurs.** L'indice `i` du robot devient `j = 4 - i` : le capteur de gauche pour le robot est celui de droite dans Godot.

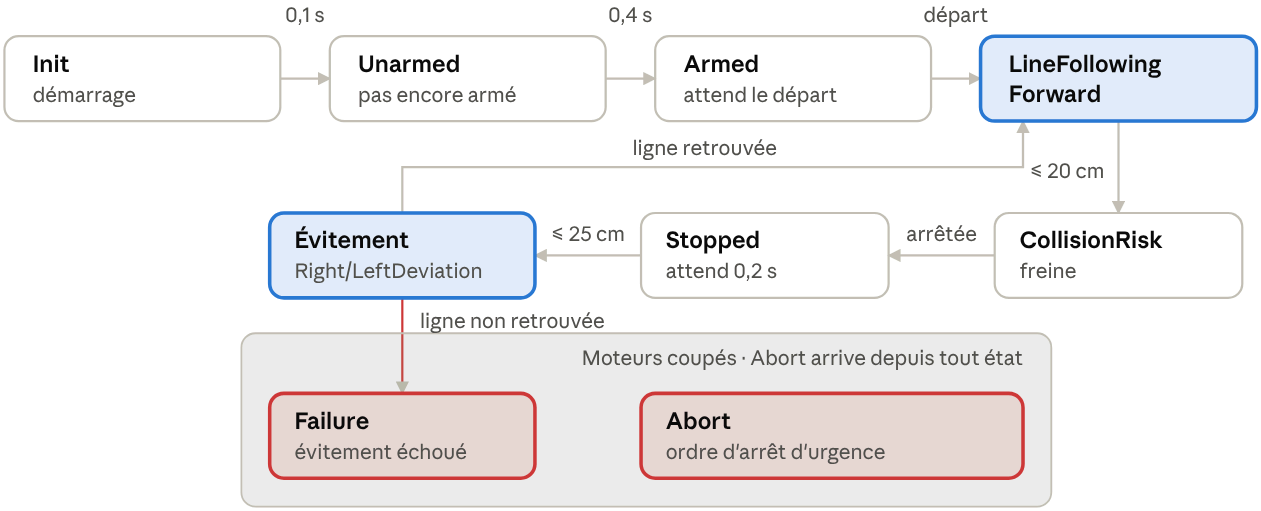
Sans ce miroir, le robot corrigeait ses erreurs dans le mauvais sens et sortait de la ligne. L'ordre des capteurs avait été perdu pendant la refonte de la machine à états ; il est rétabli dans `SimShmBridge.cpp`.

Pour le démarrage, le FSM attend un ordre sur un tube nommé (FIFO) `/tmp/peripherals.sock`, que `run_sim.sh` crée. Les réglages se modifient à chaud par le socket `/tmp/fsm.sock` (messages `SetConfigKey`, réponse ACK ou NACK) ; le site de télémétrie passe par ce chemin.

Le cerveau FSM

Le FSM est une boucle à 50 Hz : à chaque cycle de 20 ms, il lit les capteurs, applique les règles de l'état courant et publie une puissance et un angle de roues. Il a 10 états ; le schéma montre le parcours normal et l'évitement.

Le robot suit la ligne, s'arrête devant l'obstacle, puis le contourne



machine à états du FSM · 10 états

Les deux états d'évitement `RightDeviation` et `LeftDeviation` ne diffèrent que par le côté du premier virage (`AVOID_SIDE`). `Failure` est atteint si la ligne n'est pas retrouvée à la fin de l'évitement ; `Abort` vient d'un ordre d'arrêt d'urgence, qui coupe la puissance et ne repart qu'après un nouvel ordre de démarrage.

Transitions qui ne sont pas dessinées : à la barre d'arrivée, `LineFollowingForward` passe directement à `Stopped` , qui revient à `Armed` après 0,5 s. Pendant `CollisionRisk` , si l'obstacle s'éloigne de plus de 5 cm au-delà du seuil d'arrêt, le FSM reprend le suivi de ligne. Dans `Stopped` , après 0,2 s, si l'obstacle n'est plus à 25 cm ou moins, il reprend aussi le suivi de ligne.

Le suiveur de ligne pas à pas

Le suiveur calcule à chaque cycle de 20 ms un angle de roues qui ramène le ruban sous le centre du robot, puis une vitesse adaptée à cet angle.

1. **Erreur.** Barycentre pondéré des cinq valeurs de noirceur (poids −2 à +2) : 0 si le ruban est centré, négatif ou positif sinon (exemple chiffré dans « Les capteurs »).
2. **Lissage.** Moyenne exponentielle de coefficient 0,4 pour éviter les saccades, avec une zone molle de 0,15 autour de zéro où le robot ne corrige presque pas.
3. **Correction PD+I.** L'angle de roues est la somme d'un terme proportionnel (KP = 0,58), d'un terme dérivé qui amortit les oscillations (KD = 0,035) et d'un petit terme intégral qui compense un biais constant.

4. **Vitesse en virage.** Plus l'angle est grand, plus le virage est serré et plus il faut ralentir pour ne pas faire tomber la bille.
5. **Puissance.** La vitesse voulue devient une puissance : v / vitesse de croisière × `MAX_POWER_PCT` (50 %), puis le profil de vitesse limite l'accélération à 0,735 m/s².

Vitesse en virage. La bille résiste jusqu'à une accélération latérale a_max = 0,735 m/s² (voir « La bille »). Dans un virage de rayon R, l'accélération vaut v²/R : la vitesse maximale est donc v = k √(a_max · R), avec k = `CURVE_SAFETY_FACTOR` = 0,80 et R = 0,14 / tan δ.

Angle de roues	Rayon R	Vitesse maximale
Droit	très grand	0,32 m/s (croisière)
20°	0,38 m	0,32 m/s
45°	0,14 m	0,26 m/s (environ 40 % de puissance moteur)

Perte de ligne. Si plus aucun capteur ne voit le ruban, le FSM garde le dernier angle de roues et roule à la vitesse minimale de virage (`MIN_CURVE_SPEED` , 0,20 m/s), en espérant retrouver la ligne dans la courbe. Il ne s'arrête pas tout de suite.

Barre d'arrivée. Une barre noire transversale allume au moins 4 capteurs sur 5, ou une somme de noirceur d'au moins 3,6. Le FSM passe alors en `Stopped` et freine progressivement.

Point faible actuel. Depuis la simplification de la machine à états, la barre d'arrivée est parfois détectée trop tôt pendant un tour complet : le robot s'arrête à tort. Ce problème est indépendant des changements d'évitement et reste ouvert.

L'évitement d'obstacle en 12 étapes

Quand la distance lissée tombe à 20 cm ou moins, le FSM s'arrête (états CollisionRisk, puis Stopped). Si l'obstacle est encore à 25 cm ou moins après au moins 0,2 s d'arrêt, il exécute une manœuvre en boucle ouverte : il ne regarde les capteurs de ligne qu'à la toute fin. Si l'obstacle a disparu, il reprend le suivi de ligne. Chaque déplacement est une vitesse fixe (0,16 m/s, soit 25 % de puissance) multipliée par une durée : 1 s de déplacement = 16 cm.

Étape	Action	Paramètre	Valeur par défaut	Bornes
1	Reculer	`AVOID_REVERSE_DIST_M`	0,25 m	0,10 à 0,50
2	Tourner les roues du côté choisi	`AVOID_ANGLE_1_DEG`	45°	15 à 45
3	Avancer en virage	`AVOID_TIME_1_S`	1,10 s	0,3 à 3,0
4	Roues droites	—	—	—
5	Avancer en diagonale	`AVOID_DIAG_TIME_S`, `AVOID_DIAG_SPEED`	1,10 s, 0,16 m/s	0,3 à 3,0 ; 0,12 à 0,25
6	Tourner les roues de l'autre côté	`AVOID_ANGLE_2_DEG`	45°	15 à 45
7	Avancer en virage	`AVOID_TIME_2_S`	0,70 s	0,2 à 2,0
8	Roues droites, avancer parallèlement à la ligne	`AVOID_PARALLEL_TIME_S`	1,00 s	0,2 à 4,0
9	Tourner les roues vers la ligne	`AVOID_ANGLE_3_DEG`	45°	15 à 45
10	Avancer en virage	`AVOID_TIME_3_S`	0,70 s	0,2 à 2,0
11	Roues droites	—	—	—
12	Avancer jusqu'à retrouver la ligne	`AVOID_RETURN_TIME_S`, `AVOID_RETURN_SPEED`	3,5 s maximum, 0,16 m/s	1,0 à 5,0 ; 0,12 à 0,25

Le côté du premier virage vient de `AVOID_SIDE`. Les angles sont en degrés, de l'angle de roues FSM (jusqu'à 45). Allonger la durée d'un virage fait tourner davantage le robot : à 45° de roues, la voiture tourne d'environ 45° de cap sur la durée par défaut.

Où changer ces valeurs. Dans `fsm.conf` (relu au démarrage), ou à chaud depuis le site de télémétrie, section « Évitement d'obstacle (FSM) : 12 étapes ». Le FSM borne chaque valeur et réécrit la nouvelle dans le fichier. Pour que le robot avance plus longtemps

pendant le contournement, on augmente les durées (`AVOID_DIAG_TIME_S` , `AVOID_PARALLEL_TIME_S`).

Limite à retenir. Les durées supposent la vitesse de manœuvre de 0,16 m/s, soit 25 % de la puissance quand la croisière est à 0,32 m/s. Si la croisière ou `MAX_POWER_PCT` changent, il faut recalibrer. Un temps maximal global protège contre une manœuvre qui ne finit pas (14 s ou la somme des étapes, selon la plus grande).

Diagnostic. Le deuxième obstacle est à environ 0,6 m d'un coin à 90° : si le robot avance trop longtemps, il arrive trop près du coin et la manœuvre échoue du côté droit. Une vitesse de 0,20 m/s avec des durées réduites avait provoqué des collisions avec le cube : la vitesse a donc été fixée à 0,16 m/s.

La bille

Le PiCar transporte une bille dans une coupelle conique très aplatie, et le projet est réussi si elle reste dedans : la bille tombe dès que l'accélération du robot dépasse a_max = 0,735 m/s², d'où tous les profils de vitesse du FSM.

La coupelle a un rayon de 20 mm et une profondeur de 1,5 mm, donc une pente tan θ = 1,5 / 20 = 0,075. La bille sort quand l'accélération dépasse l'accélération critique :

```
a_{max} = g \tan\theta = 9{,}81 \times 0{,}075 \approx 0{,}735\ \text{m/s}^2
```

En simulation, `ball_monitor.gd` déclare la bille tombée quand elle s'éloigne de plus de 20 mm du centre de la coupelle.

Le défaut corrigé. La bille tombait dès que le robot atteignait la barre d'arrivée. Dans l'état `Stopped` , la puissance était coupée d'un seul coup : la vitesse passait de 0,32 m/s à zéro en un cycle de 20 ms, soit environ 16 m/s², plus de vingt fois la limite.

	Avant la correction	Après la correction
Fin de parcours	Coupure instantanée	Rampe du profil de freinage
Décélération	Environ 16 m/s²	Au plus 0,735 m/s²
Durée de l'arrêt	20 ms	Environ 0,43 s, quelques centimètres après la barre

	Avant la correction	Après la correction
Bille tombée en simulation	2 essais sur 3	0 essai sur 3

La fonction `must_cut_power` réserve maintenant la coupure immédiate aux arrêts d'urgence et aux états d'échec (`Failure`, `Abort`). Un test automatisé vérifie que la puissance ne baisse jamais de plus d'environ 2,3 points par cycle.

Une puissance maximale plus élevée (`MAX_POWER_PCT` au-dessus de 50 %) augmente le risque pour la bille ; ce réglage n'a pas été testé.

Le mode réaliste

Par défaut, la simulation est parfaite, ce qui rend les réglages trop optimistes ; le mode réaliste (`realism.gd`) ajoute les défauts attendus du robot réel. On l'active avec la touche `R` ou la variable `REALISM=1`, et `REALISM_SEED` fixe le hasard pour rejouer un parcours.

Défaut simulé	Valeur	Équivalent réel
Biais de direction	4°	Roues mal alignées : le robot tire d'un côté
Jeu du servo	2°	Le servo de direction a du jeu mécanique
Gain moteur	93 %	Le moteur fournit moins que la vitesse demandée
Latence	60 ms	Délai entre la commande et son effet
Chocs aléatoires	Occasionnels	Petites perturbations de la trajectoire
Bruit ultrason	Variable	Mesure de distance instable
Erreurs de ligne	Occasionnelles	Capteur de ligne qui se trompe

Problème ouvert. Avec le mode réaliste, le robot perd la ligne dès le premier virage de la piste, même sans obstacle. Le suivi de ligne est en cause : ni une vitesse réduite ni les variantes de gains essayées ne règlent le problème. Rien n'a encore été comparé au comportement du robot réel.

Tests et résultats

Tests rapides. Godot peut tourner sans fenêtre (`--headless`) et être piloté par des scripts qui relevent la position du robot. Pour aller plus vite que le temps réel, il faut accélérer ensemble Godot (`Engine.time_scale = N` et `physics_ticks_per_second = 60 × N`) et la boucle du FSM, avec le même pas de temps logique. Le temps simulé se calcule avec les pas de physique, pas avec les images affichées.

Vitesse	Durée d'un tour de 53 s	Fiabilité
1×	53 s	Référence
5×	Environ 16 s	Correcte
8×	—	Le FSM ne suit plus

Les résultats ne sont pas parfaitement reproductibles d'une exécution à l'autre. Ces scripts de test ne font pas encore partie du dépôt.

Tests unitaires. Neuf cibles de test sont lancées par `./build.sh --test`. Les tests du navigateur du FSM couvrent le ralentissement à l'approche, la séquence des 12 étapes (ordre, angles, vitesses, durées à un cycle près), les bornes de réglage, l'arrêt progressif et la conversion vitesse-puissance. Le client de télémétrie a aussi un test de l'arrêt d'urgence (deux ordres : FSM puis moteurs). Un seul test échoue : `test_telemetry`, instable et déjà en échec avant ces changements.

Bugs corrigés.

Symptôme	Cause	Correction
Le robot ne démarre pas	Tube de démarrage plus créé	`run_sim.sh` crée le FIFO
Évitement aléatoire	FSM à 100 Hz, gains réglés pour 50 Hz	`HZ=50` dans `fsm.conf`
Côté de contournement ignoré	`AVOID_SIDE` n'était plus lue	Lecture rétablie
Robot qui se coince contre le cube	Ralentissement à l'approche disparu	Rétabli, test unitaire
Corrections dans le mauvais sens	Ordre des capteurs plus inversé	Miroir `j = 4 - i` rétabli

Symptôme	Cause	Correction
Le robot repart après l'arrêt d'urgence	Ordre envoyé seulement aux moteurs	FSM puis moteurs, passage en `Abort`
Bille qui tombe à l'arrivée	Coupure instantanée de la puissance	Arrêt progressif

État actuel. Obstacle 1 : réussi 4 fois sur 4. Obstacle 2 : réussi sans mode réaliste. Mode réaliste et tour complet : voir les problèmes ouverts ci-dessus. Rien n'est validé sur le robot réel.

Dépannage par symptôme

Symptôme	Cause probable	Que faire
Godot ne se connecte pas	SimBridge n'écoute pas sur le port 9000	Relancer `./run_sim.sh` et attendre les messages de démarrage
Le robot ne bouge pas après le démarrage	Tube `/tmp/peripherals.sock` absent, ou ordre de démarrage non reçu	Vérifier que le FIFO existe, relancer `run_sim.sh`
`run_sim.sh` refuse de démarrer	Un fichier ordinaire porte le nom du tube	Supprimer ce fichier dans WSL, puis relancer
Le robot sort de la ligne dès le départ	Capteurs ou direction inversés	Vérifier le miroir `j = 4 - i` et l'inversion de direction dans SimBridge
L'évitement réussit puis échoue au hasard	FSM pas à 50 Hz	Vérifier `HZ=50.0` dans `fsm.conf`
Le robot se colle au cube	Pas de ralentissement à l'approche	Vérifier les seuils de distance (45 cm, 25 cm, 20 cm)
La bille tombe en virage	Vitesse trop haute ou `MAX_POWER_PCT` trop grand	Remettre 50 %, vérifier `CURVE_SAFETY_FACTOR`
Le robot s'arrête en plein parcours	Fausse détection de la barre d'arrivée	Problème ouvert depuis la refonte de la machine à états

Symptôme	Cause probable	Que faire
La simulation s'arrête avec « Caught signal 15 »	Un autre script a tué SimBridge (par exemple `pkill`)	Ne pas lancer de scripts de test pendant qu'on utilise la simulation
Le robot repart après l'arrêt d'urgence	Cas corrigé : le FSM passe en `Abort`	Armer de nouveau pour repartir
Un réglage change sans effet	Valeur hors bornes, remplacée par la borne	Lire la réponse ACK ou NACK du FSM et les bornes de la section « Évitement »

Glossaire

Terme	Sens dans ce projet
Ackermann	Modèle de direction où seules les roues avant tournent ; la voiture suit un cercle de rayon L / tan δ
Barycentre	Moyenne des positions des capteurs pondérée par leur noirceur ; indique où est le ruban
Boucle ouverte	Manœuvre commandée par des durées, sans regarder les capteurs (l'évitement)
EMA	Moyenne exponentielle : lisse une mesure en mélangeant la nouvelle valeur à l'ancienne
FIFO	Tube nommé du système de fichiers, utilisé ici pour l'ordre de démarrage
FSM	Machine à états finis : le service qui décide ce que fait le robot
IPC	Communication entre processus (mémoire partagée, sockets Unix)
Médiane	Valeur du milieu d'une liste triée ; ignore une mesure aberrante isolée
PD+I	Correcteur : proportionnel (erreur), dérivé (amortit), intégral (biais constant)

Terme	Sens dans ce projet
RayCast	Rayon invisible de Godot qui renvoie le premier objet touché
RigidBody	Objet de Godot soumis à la physique (la bille)
Scene / nœud	Fichier Godot contenant un arbre d'objets / un objet de cet arbre
SimBridge	Service qui relie Godot (TCP) à la mémoire partagée des services C++
Zone molle	Plage d'erreur proche de zéro où le suiveur corrige peu

Annexes

Fichiers qui comptent

Fichier	Rôle
`godot/main.tscn`	La scène : sol, circuit, voiture, bille, obstacles
`godot/picar_raycast.gd`	Déplacement d'Ackermann, collisions, animation des roues
`godot/line_follower_raycast.gd`	Les cinq capteurs de ligne (15 rayons chacun)
`godot/ball_monitor.gd`	Surveille la chute de la bille
`godot/realism.gd`	Défauts du mode réaliste
`services/run_sim.sh`	Lance SimBridge et le FSM, crée le FIFO
`services/SimBridge/src/SimShmBridge.cpp`	Pont Godot et mémoire partagée, conversions miroir
`services/FSM/src/FSM.cpp`	Cycle principal, lecture des réglages
`services/FSM/src/FsmNavigator.cpp`	Vitesse, puissance, ralentissement, arrêt
`services/FSM/src/ObstacleAvoidance.cpp`	Les 12 étapes de l'évitement

Fichier	Rôle
`services/FSM/inc/AvoidanceSettings.hpp`	Clés `AVOID_*` et leurs bornes
`fsm.conf`	Réglages lus au démarrage

Réglages actuels de `fsm.conf` (hors clés `AVOID_*`, décrites plus haut)

Clé	Valeur
`HZ`	50,0
`MAX_ACCELERATION`	0,735
`STRAIGHT_CRUISE_SPEED`	0,32
`MAX_POWER_PCT`	50
`MIN_CURVE_SPEED`	0,20
`CURVE_SAFETY_FACTOR`	0,80
`LINE_KP`	0,58
`LINE_KD`	0,035
`OBSTACLE_STOP_DIST_CM`	20
`REVERSE_SPEED`	0,12 (sans effet sur l'évitement)
`AVOID_SIDE`	1,0

Ce que la simulation ne reproduit pas

- L'adhérence réelle des roues, le glissement et les vibrations du sol.
- La réponse exacte du moteur et de la batterie : la simulation utilise v = (puissance / 50) × 0,32 m/s, à vérifier sur le robot.
- Le comportement réel du capteur ultrason (rebonds, faux échos).
- La lumière ambiante et la réflectivité du ruban pour les capteurs de ligne.

Tous les résultats de ce document viennent de la simulation ; le robot réel n'a pas été testé avec ces réglages.